abstraction.py Documentation

State Encoding and Hand Abstraction Reference

Source file: c:\Users\PRABAL YADAV\Desktop\machine learning iim\pokerbot bestmodelcopy 6people\abstraction.py

Generated: 2025-12-24 14:39

This document explains every public helper in abstraction.py and lists all game-state fields required by encode_state with examples.

Module overview

abstraction.py centralizes all poker hand abstraction helpers, including card utilities, Treys evaluators, hand-strength lookup tables (LUT), draw and board texture helpers, bucketizers, and the main encode_state feature encoder used for Deep CFR style agents.

Note: the file contains an older, commented block near the top. The active implementation begins after the visible divider line that starts with 'File overview' and the real imports.

Runtime side effects

On import, _load_or_build_lut() is called. If hand_strength_lut.pkl is missing, it will build a LUT by sampling random 7-card hands. This can take time and will create the file on disk.

Card encoding and utilities

Cards are encoded as integers 0..51 using the standard 52-card mapping: 0-12 = 2..A of spades, 13-25 = 2..A of hearts, 26-38 = 2..A of diamonds, 39-51 = 2..A of clubs.

```
card_rank(card) -> int in [2..14]
card_suit(card) -> int in [0..3] where 0=S,1=H,2=D,3=C
```

Examples:

```
card_rank(0) = 2
card_suit(0) = 0 (spades)
card_rank(51) = 14 (ace)
card_suit(51) = 3 (clubs)
```

_card_0_51_to_treys converts the 0..51 id into Treys Card.new('As') style encodings.

```
_card_0_51_to_treys(51) -> Treys int for Ac
```

Hand evaluation and strength LUT

evaluate_5card and evaluate_7card wrap Treys Evaluator. Treys returns smaller numbers for stronger hands; this code negates the score so higher is better. For fewer than 5 cards, evaluate_7card returns 0.

```
evaluate_5card([0, 5, 13, 22, 44]) -> int score
evaluate_7card([12, 25], [1, 14, 27, 40, 48]) -> int score
```

_build_strength_lut samples random 7-card hands and maps evaluate_7card() scores to percentile values in [0,1]. _load_or_build_lut loads the cached mapping from hand_strength_lut.pkl if present.

normalized_strength(hole, board) returns a continuous [0,1] strength: preflop uses a simple rank heuristic, postflop uses the LUT percentile.

```
normalized_strength([51, 0], []) -> 0.57 (example)
normalized_strength([51, 0], [3, 18, 31]) -> 0.73 (example)
```

Board texture and draw helpers

_board_texture(board) returns three floats: highness (0..1), connectedness (0..1), suitedness (0..1).

```
board = [0, 9, 18]  # 2s, Js, 7h
_board_texture(board) -> (highness, connectedness, suitedness)
```

_has_flush_draw(cards) checks if 4 or more cards share a suit. _has_straight_draw(cards) checks for any 4-card straight window. _hand_class returns the Treys class string for a 5+ card set.

Strength buckets

Buckets provide coarse strength signals per street. Each bucket is a 3-element one-hot list.

```
_preflop_bucket(hole) -> [strong, medium, weak]
_flop_bucket(hole, board) -> [air, draw, made]
_turn_river_bucket(hole, board) -> [weak, medium, strong]
_street_strength_bucket(hole, board, street) -> one of the above
```

Examples:

```
_preflop_bucket([51, 50]) -> [1, 0, 0]  # AA
_flop_bucket([10, 23], [5, 18, 31]) -> [0, 1, 0]  # draw
_turn_river_bucket([12, 25], [1, 14, 27, 40, 48]) -> [0, 0, 1]  # strong
```

encode_state: required GameState fields

encode_state expects a PokerEnv GameState-like object. These fields must be present and consistent for correct encoding.

Field	Type	Example	Notes
street	int	1	0=pre,1=flop,2=turn,3=river
num_players	int	6	total seats
button_player	int	2	dealer/button seat index
sb_player	int	3	small blind seat index
bb_player	int	4	big blind seat index
pot	float	12.0	total pot
current_bet	float	4.0	current bet to call
contrib	List[float]	[2,4,0,4,4,0]	per-seat invested this street
stacks	List[float]	[96,98,0,80,90,100]	current stacks
initial_stacks	List[float]	[100,100,100,100,100,100]	reference for normalization
folded	List[bool]	[F,F,T,F,F,F]	folded status per seat
players_acted	List[bool]	[F,T,T,F,F,T]	acted this street
to_act	int	0	seat to act or -1/None
last_aggressor	int	3	last raiser or -1/None
board	List[int]	[0, 9, 18]	community cards 0..51
hole	List[List[int]]	[[51,0], ...]	hole cards per player
action_seq	List[Tuple]	[(3,4,0.5), ...]	recent actions

All numeric fields are expected to be floats or ints, not tensors. Lists should be length num_players.

encode_state: feature layout and formulas

encode_state returns a 1D FloatTensor. For N seats and K=ACTION_SEQ_LEN, total length is 30 + 8N + 7K. With N=6 and K=6, length = 120.

```
Order:
1) street_one_hot (4)
2) hero_position_one_hot (N)
3) public scalars (10)
4) strength bucket (3)
5) range/pressure features (9)
6) per-seat features (N * 7)
7) hole_feats (4)
8) action_seq features (K * 7)
```

Public scalars (10, in order): pot_norm, to_call_norm, pot_after_call_norm, curr_bet_norm, spr_norm, hero_is_sb, hero_is_bb, last_agg_present, last_agg_rel_norm, board_strength.

```
pot_norm = pot / (ref_stack * max(2, num_players))
ref_stack = max(initial_stacks) if available else STACK_SIZE
to_call = max(0, current_bet - contrib[hero])
to_call_norm = to_call / ref_stack
pot_after_call_norm = (pot + to_call) / (ref_stack * max(2, num_players))
curr_bet_norm = current_bet / ref_stack
spr_raw = stacks[hero] / max(1, pot)
spr_norm = min(spr_raw, 20.0) / 20.0
last_agg_rel_norm = ((hero - last_aggressor) % num_players) / (num_players - 1)
```

encode_state: range and pressure features

range_advantage is a proxy derived from board texture. It uses board_highness, connectedness, and suitedness. If the last aggressor is hero, range_advantage = base_adv, else 1 - base_adv.

hero_polarized and villain_polarized are flags when the last aggressor bet big (size_norm >= 0.6) at low SPR (spr_raw <= 3.0).

fold_equity_proxy and pot_pressure are computed against a primary villain (to_act if not hero, else next live seat).

Blocker flags: hero_blocks_top_pair, hero_blocks_nut_flush, hero_has_high_card_on_flush_board.

line_consistency = aggressive_actions / opportunities from action_seq.

encode_state: per-seat features

Per-seat features are rotated so the hero is first. For each seat, encode the following 7 values in order:

```
stack_norm, contrib_norm, to_call_norm, folded_flag, all_in_flag, acted_flag, to_act_flag
```

Where: stack_norm = stacks[pid]/ref_stack, contrib_norm = contrib[pid]/ref_stack, to_call_norm = max(0, current_bet - contrib[pid]) / ref_stack.

encode_state: action_seq encoding

action_seq stores recent actions as tuples: (player, action, size_norm). It is built in poker_env._record_action with size_norm = min(invested / pot_before, 4.0) / 4.0.

encode_action_sequence encodes the most recent K actions (latest first). For each action it emits 7 values:

```
actor_offset, is_fold, is_check, is_call, is_raise_small, is_raise_big, size_norm
```

actor_offset = ((actor - hero) % num_players) / (num_players - 1). Raises treat ACTION_RAISE_MEDIUM and ACTION_ALL_IN as 'big'.

```
Example action_seq: [(3, ACTION_RAISE_MEDIUM, 0.50), (4, ACTION_CALL, 0.10), (0, ACTION_CALL, 0.10)]
```

Worked example (partial)

Assume N=6, hero=0, button=2, street=flop, pot=12, current_bet=4, contrib[hero]=2, stacks[hero]=96, initial_stacks all 100.

```
ref_stack = 100
pot_norm = 12 / (100 * 6) = 0.020
to_call = 4 - 2 = 2
```

```
to_call_norm = 0.02
pot_after_call_norm = 14 / 600 = 0.0233
spr_raw = 96 / 12 = 8.0
spr_norm = 8 / 20 = 0.40
hero_pos = (0 - 2) % 6 = 4 -> hero_pos_one_hot[4] = 1
```

These values are placed in the vector in the order described above, followed by the per-seat features, hole_feats, and the action_seq encoding.